

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto: GS Stock & Facturación.

Link del documento de software:

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Link al documento:

[Sprint2.docs](#)

Link de la presentación: [Ir a presentacion](#)

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

Product Backlog

Historias de usuario:

→ [Ir a historias de usuario](#)

Lista de tareas

Configurar servidor Express con Node.js (Finalizada)

- 1.1 Inicializar proyecto con npm (Finalizada)
- 1.2 Configurar estructura de carpetas (Finalizada)
- 1.3 Crear servidor básico con Express (Finalizada)

Configurar Vue.js y estructura del frontend (Finalizada)

- 2.1 Crear proyecto Vue con Vite (Finalizada)
- 2.2 Definir estructura base de carpetas (components, views, router) (Finalizada)

Configurar contenedor Docker del proyecto (Finalizada)

- 3.1 Crear Dockerfile para backend (Finalizada)
- 3.2 Crear Dockerfile para frontend (Finalizada)
- 3.3 Crear docker-compose.yml para levantar todo junto (Finalizada)

Configurar entorno de pruebas (Finalizada)

- 4.1 Crear archivos .env para desarrollo y producción (Finalizada)
- 4.2 Conectar base de datos de pruebas (Finalizada)
- 4.3 Configurar comandos y scripts de test (Finalizada)

Implementar autenticación con JWT (Finalizada)

- 5.1 Crear modelo de usuario (con email y contraseña) (Finalizada)
- 5.2 Crear endpoint de login que genere JWT (Finalizada)
- 5.3 Crear middleware para validar token (Finalizada)

Implementar cifrado de contraseñas (bcrypt) (Finalizada)

- 6.1 Hashear contraseña antes de guardar en base de datos (Finalizada)
- 6.2 Validar contraseña hasheada al iniciar sesión (Finalizada)

Crear endpoints de login y logout (Finalizada)

- 7.1 POST /api/login con validación de datos (Finalizada)

7.2 POST /api/logout (Finalizada)

Implementar sistema de recuperación de contraseña

8.1 Diseñar modelo de token temporal con expiración (Finalizada)

8.2 Crear endpoint para solicitar recuperación (envía token por correo o devuelve para test)

8.3 Crear endpoint para cambiar contraseña con token válido

8.4 Validar contraseña nueva y vencimiento del token (Finalizada)

8.5 Crear UI para solicitar recuperación (Finalizada)

8.6 Crear UI para cambiar contraseña con token (Finalizada)

Implementar CRUD de usuarios (Finalizada)

9.1 Modelo: nombre, correo, contraseña, rol, estado (Finalizada)

9.2 Endpoint POST /usuarios para crear usuario (Finalizada)

9.3 Endpoint GET /usuarios para listar usuarios (Finalizada)

9.4 Endpoint PUT /usuarios/:id para actualizar (Finalizada)

9.5 Endpoint DELETE /usuarios/:id para eliminar o desactivar (Finalizada)

Asignar roles y permisos dinámicamente (Finalizada)

10.1 Crear estructura de roles en base de datos (Finalizada)

10.2 Lógica para asignar roles al crear o editar usuario (Finalizada)

10.3 Middleware para verificar rol en rutas protegidas (Finalizada)

10.4 Validación para impedir que usuarios cambien su rol o el de otros sin permisos (Finalizada)

Crear UI para gestión de usuarios (Finalizada)

11.1 Tabla con lista de usuarios (Finalizada)

11.2 Botones para editar, eliminar y asignar rol (Finalizada)

11.3 Formulario de edición y creación (Finalizada)

11.4 Selector/desplegable para rol (Finalizada)

11.5 Conectar UI al backend con axios

11.6 Mostrar mensajes de éxito/error (Finalizada)

Creación de lógica para invalidar token al hacer logout (Finalizada)

12.1 Eliminar token del frontend (Finalizada)

Implementar frontend para creación de usuarios (Finalizada)

13.1 Crear formulario con validación (Finalizada)

13.2 Enviar datos al backend mediante POST

13.3 Mostrar mensajes de respuesta

Crear los endpoints de envío de datos de creación de usuario a la base

14.1 Endpoint POST /usuarios (Finalizada)

14.2 Validaciones de campos (Finalizada)

14.3 Guardar en base de datos

Creación de base de datos dentro de Docker (Finalizada)

15.1 Añadir contenedor de PostgreSQL (Finalizada)

15.2 Configurar volumen (Finalizada)

15.3 Asegurar conexión desde backend (Finalizada)

Implementar CRUD de productos

16.1 Modelo: nombre, código, precio, stock

16.2 POST /productos

16.3 GET /productos

16.4 PUT /productos/:id

16.5 DELETE /productos/:id

Implementar historial de movimientos en inventario

17.1 Modelo: tipo (entrada/salida), fecha, producto, cantidad

17.2 Endpoint GET /movimientos

17.3 Registrar movimiento en cada modificación de stock

Crear UI para gestión de inventario

18.1 Tabla con productos y stock

18.2 Botones para editar producto y ver historial

18.3 Crear formulario de producto

Implementar WebSockets para actualización en tiempo real

19.1 Configurar socket.io en backend

19.2 Emitir eventos en cambios de inventario

19.3 Escuchar desde frontend y actualizar UI

Actualizar stock automáticamente tras ventas y pedidos

20.1 Crear lógica para descontar stock al crear pedido

20.2 Validar stock disponible antes de confirmar

Implementar sincronización en tiempo real con la base de datos

21.1 Asegurar consistencia con WebSockets

21.2 Manejar colisiones concurrentes si es necesario

Implementar generación de facturas

22.1 Modelo: cliente, productos, precios, total, fecha

22.2 Endpoint POST /facturas

22.3 Guardar en base de datos

Implementar envío de facturas por email

23.1 Integrar servicio SMTP o API externa

23.2 Enviar PDF generado a email del cliente

Implementar historial de facturación

24.1 Endpoint GET /facturas

24.2 Mostrar lista filtrable por fechas o cliente

Crear UI para facturación

25.1 Formulario de factura

25.2 Tabla de productos con subtotal

25.3 Botón de generar y enviar factura

Implementar seguimiento de cuentas por cobrar

26.1 Modelo: cliente, monto, vencimiento, estado

26.2 Endpoints de consulta y actualización

Crear reportes financieros

27.1 Totales por mes, por cliente, por producto

27.2 Endpoint GET /reportes/finanzas

27.3 Calcular ganancias y pérdidas

Generar informes automatizados

28.1 Crear tarea periódica (cron job o backend loop)

28.2 Enviar informes por correo o guardar en base

Implementar clasificación de clientes (A, B, C, D)

29.1 Añadir campo "clasificación" a modelo

29.2 Crear lógica de asignación manual o automática

Implementar reportes de ventas

30.1 Endpoint GET /reportes/ventas

30.2 Agrupar por fechas, productos, clientes

Implementar reportes de inventario

31.1 Endpoint GET /reportes/inventario

31.2 Agrupar por stock bajo, más vendidos

Implementar reportes financieros

32.1 Endpoint GET /reportes/finanzas

32.2 Mostrar ingresos/egresos por periodo

Creación a PDF de reportes

33.1 Usar librería para exportar HTML a PDF

33.2 Descargar desde frontend

Implementar descarga de reportes

34.1 Agregar botón de descarga PDF

34.2 Proteger con permisos si es necesario

Crear UI para visualización de reportes

35.1 Dashboard con tarjetas y gráficas

35.2 Filtros por fechas, clientes, categorías

Implementación de pedidos

36.1 Modelo: cliente, productos, total, estado

36.2 Endpoint POST /pedidos

36.3 Actualizar inventario tras pedido

Implementación de descuentos

37.1 Modelo: tipo, porcentaje, condiciones

37.2 Aplicar descuento en facturación

Crear UI para creación de pedidos

38.1 Formulario con búsqueda de productos

38.2 Resumen y total del pedido

Pila del Sprint

Historias de usuario

→ [Ir a historias de usuario](#)

Puntos de historia

Clave	Resumen	Tipo de incidencia	Épico	Estado	Persona asignada	Puntos de
GSS-31	SRC: Diseñar modelo de token temporal para recuperación	☑ Tarea		TERMINADO	FR	2
GSS-32	SRC: Crear endpoint para solicitar recuperación (enviar correo o código temporal)	☑ Tarea		TERMINADO	FR	2
GSS-33	SRC: Crear endpoint para cambiar contraseña con token válido	☑ Tarea		TERMINADO	FR	2
GSS-34	SRC: Proteger con validaciones (expiración del token, longitud mínima en contraseña, e...	☑ Tarea		TERMINADO	FR	2
GSS-35	SRC: Crear UI para solicitar recuperación de contraseña	☑ Tarea		TERMINADO	HB	3
GSS-36	SRC: Crear UI para cambiar la contraseña	☑ Tarea		TERMINADO	HB	3
GSS-37	GU: Crear endpoint para crear un nuevo usuario	☑ Tarea		TERMINADO	JL	2
GSS-38	GU: Crear endpoint para ver lista de usuarios	☑ Tarea		TERMINADO	HB	2
GSS-39	GU: Crear endpoint para actualizar datos de un usuario	☑ Tarea		TERMINADO	JL	2
GSS-40	GU: Crear endpoint para eliminar/desactivar usuario	☑ Tarea		TERMINADO	JL	3
GSS-41	GU: Crear lógica para asignar rol al crear usuario o actualizarlo	☑ Tarea		TERMINADO	JL	3
GSS-42	GU: Crear UI para gestión de usuarios	☑ Tarea		TERMINADO	GC	5
GSS-43	GU: Diseñar interfaz para ver la lista de usuarios.	☑ Tarea		TERMINADO	GC	2
GSS-44	GU: Agregar botones de acciones (editar, eliminar, asignar rol)	☑ Tarea		TERMINADO	GC	3
GSS-45	GU: Crear UI del formulario para registro/edición de usuarios	☑ Tarea		TERMINADO	GC	3
GSS-47	Extra: Pasar login UI de .HTML a .VUE	☑ Tarea		TERMINADO	HB	2
GSS-48	Dividir el github en 2 repositorios (Back y Front)	☑ Tarea		TERMINADO	HB	1

Tareas para el Sprint 2

- Definir modelo de token temporal para recuperación de contraseña
- Diseñar modelo de usuario
- Crear endpoint para solicitar recuperación
- Crear endpoint para registrar usuario
- Crear endpoint para cambiar contraseña con token
- Crear endpoint para listar usuarios
- Agregar validaciones de seguridad a la recuperación
- Crear endpoint para actualizar usuario
- Crear endpoint para eliminar o desactivar usuario
- Diseñar estructura de roles
 - Implementar lógica para asignar roles
 - Agregar middleware para proteger endpoints sensibles
- Validar que usuarios sin permiso no cambien su rol ni el de otros
- Crear interfaz para solicitar recuperación de contraseña
- Crear interfaz para cambio de contraseña
- Diseñar interfaz para ver la lista de usuarios
- Agregar botones para editar, eliminar y asignar rol
- Crear formulario para registro y edición de usuario
- Diseñar UI para asignación de roles
- Conectar frontend con backend usando fetch

Calendario de planificación

Fecha	Tarea
09/04/2025	- Definir modelo de token temporal para recuperación de contraseña - Diseñar modelo de usuario
10/04/2025	- Crear endpoint para solicitar recuperación - Crear endpoint para registrar usuario
11/04/2025	- Crear endpoint para cambiar contraseña con token - Crear endpoint para listar usuarios
12/04/2025	- Agregar validaciones de seguridad a la recuperación - Crear endpoint para actualizar usuario
13/04/2025	- Crear endpoint para eliminar o desactivar usuario - Diseñar estructura de roles
14/04/2025	- Implementar lógica para asignar roles - Agregar middleware para proteger endpoints sensibles
15/04/2025	- Validar que usuarios sin permiso no cambien su rol ni el de otros
16/04/2025	- Crear interfaz para solicitar recuperación de contraseña
17/04/2025	- Crear interfaz para cambio de contraseña
18/04/2025	- Diseñar interfaz para ver la lista de usuarios - Agregar botones para editar, eliminar y asignar rol
19/04/2025	- Crear formulario para registro y edición de usuario
20/04/2025	- Diseñar UI para asignación de roles - Conectar frontend con backend usando fetch
21/04/2025	- Presentación

Resultados del Sprint

- Resultado de cumplimiento de las tareas

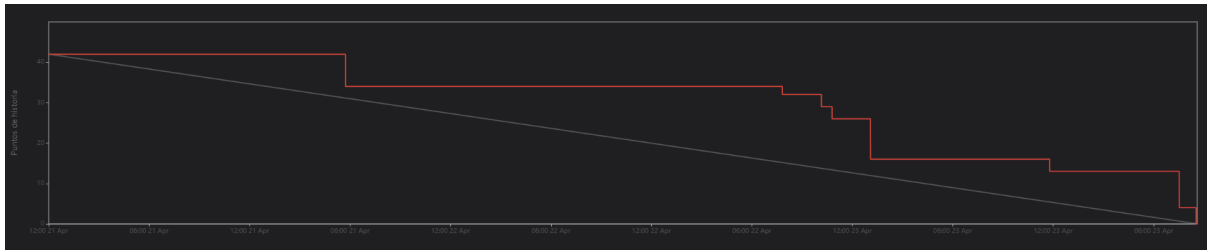
Durante este sprint se lograron cumplir la mayoría de las tareas propuestas al inicio, ya que se completaron 14 de las 17 tareas planificadas. Dos tareas quedaron en estado de "en progreso" porque no se finalizaron debido a limitaciones de tiempo, y una tarea permaneció en estado "planificada", sin haberse iniciado, por la misma razón. Esto se debió a que durante la Semana Santa el equipo no trabajó, lo que generó una mayor carga de trabajo hacia el final del sprint.

- Listar tareas concluidas
 - Diseñar modelo de token temporal para recuperación
 - Crear endpoint para solicitar recuperación

- Crear endpoint para cambiar contraseña con token válido
- Proteger con validación (expiración del token, longitud mínima en contraseña, etc.)
- Crear UI para solicitar recuperación de contraseña
- Crear UI para gestión de usuarios
- Crear UI para cambiar la contraseña
- Diseñar interfaz para ver la lista de usuarios
- Agregar botones de acciones (editar, eliminar, asignar rol)
- Crear UI del formulario para registro/edición de usuario
- Crear endpoint para crear un nuevo usuario
- Crear endpoint para ver lista de usuarios
- Crear endpoint para actualizar datos del usuario
- Crear endpoint eliminar/desactivar usuario
- Listar tareas en proceso
 - Crear endpoint para ver lista de usuarios
 - Crear endpoint eliminar/desactivar usuario
- Lista de tareas que se planificaron y no se pudieron concluir
 - Manejar middleware con permisos de usuarios según su rol
 - Conectar endpoint de guardado de datos al crear usuario
 - Conectar endpoint con la interfaz de recuperación de contraseña.

Métricas del Sprint

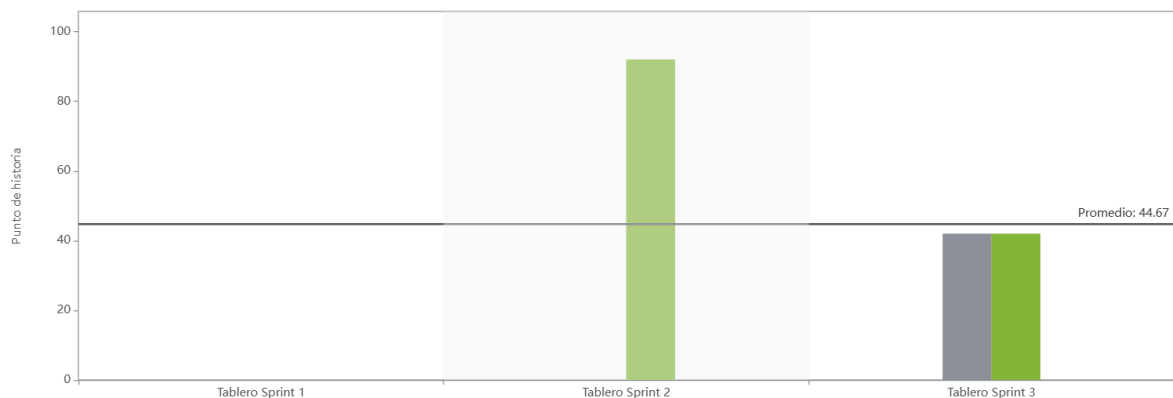
Gráfico Burndown



La gráfica burndown para este Sprint 2 nos dice que tuvimos un inicio bastante bueno, seguíamos más o menos el ritmo esperado en los primeros días. Sin embargo, cuando llegó Semana Santa, prácticamente todo se detuvo. Se ve clarísimo en ese tramo horizontal largo donde no avanzamos nada, y es que la mayoría del equipo estaba ausente o con disponibilidad muy limitada durante esos días festivos.

Ya para la segunda parte del sprint retomamos el trabajo, pero no de forma constante sino a "jalones", como se ve en esos escalones en la gráfica. Fuimos completando tareas en bloques concentrados, intentando recuperar el tiempo perdido. Al final logramos terminar bastante trabajo, aunque claramente no seguimos el ritmo ideal que marca la línea gris.

Métrica de velocidad



La gráfica de velocidad de Jira muestra que, durante el Sprint 3, el equipo cumplió exactamente con la cantidad de puntos de historia que había planificado al inicio. Aunque la cantidad de puntos completados fue menor en comparación con el sprint anterior, esto indica una estimación realista y un buen control del trabajo asignado, demostrando consistencia en el cumplimiento de los compromisos asumidos.

Indicador numérico del éxito del sprint

Nota: 8.2/10

Justificación:

En este segundo sprint se completaron 14 de las 17 tareas que estaban planificadas, lo que representa un avance del 82%. Entre lo que se logró terminar, se incluyen funciones importantes como la recuperación de contraseña, el sistema CRUD para usuarios, y la

interfaz de gestión de usuarios. Esto muestra un avance sólido en funcionalidades clave del proyecto.

Quedaron pendientes dos tareas que se estaban trabajando (como el endpoint para listar usuarios), y una que ni siquiera se pudo comenzar (la asignación dinámica de roles). Esto se debió principalmente a la falta de tiempo, ya que parte del equipo tuvo menos disponibilidad durante la Semana Santa.

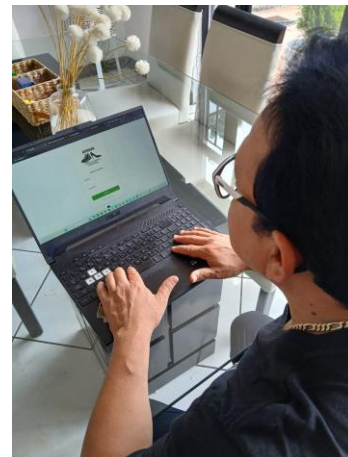
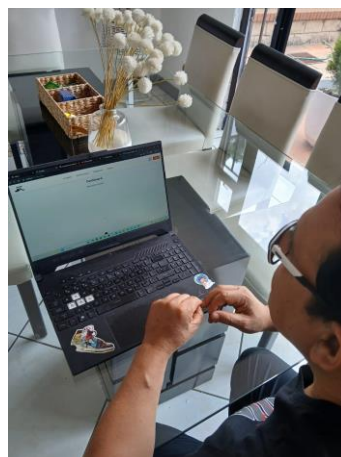
A pesar de eso, las tareas que sí se finalizaron fueron bien validadas y probadas, incluyendo aspectos importantes como el cifrado de contraseñas y el control de acceso por roles. La carga de trabajo se acumuló hacia el final del sprint, lo que dificultó completar todo. Por eso, la nota es un poco menor en comparación con el sprint anterior (que fue 8.5).

Discusión del éxito del sprint

El Sprint 2 ha tenido un buen rendimiento, hemos completamos las tareas dentro del tiempo estipulado. La gráfica de velocidad muestra un cumplimiento bueno de las tareas en el backlog, especialmente en las funcionalidades críticas como la configuración de endpoints que nos servirán más adelante y la creación de pantallas necesarias.

En base a la gráfica burndown indica que el manejo del tiempo no fue completamente eficiente, ya que la estipulación que da el burndown no es igual al seguimiento de las tareas, se ve en el gráfico que algunas tareas se mantuvieron sin completar en cierto tiempo por lo que la grafica muestra una linea recta en ciertas partes y al final se terminaron algunas tareas ya en la fecha final.

Evidencia de muestra del incremento desarrollado a usuarios finales



Durante la revisión de este Sprint, presentamos al Product Owner las actualizaciones implementadas, incluyendo el login actualizado con el logo corporativo que había solicitado en el Sprint anterior. También mostramos la nueva interfaz gráfica para gestión de usuarios, que permite crear usuarios y editar sus estados (activo/inactivo). El Product Owner expresó gran satisfacción con el diseño y la usabilidad, mencionando

específicamente que estaba "bastante contento" con la apariencia general del proyecto.

Adicionalmente, presentamos la interfaz para recuperación de contraseñas, aunque aclaramos que esta función aún no está operativa y ha sido reprogramada para el próximo Sprint debido a limitaciones de tiempo. A pesar de este aplazamiento, el Product Owner no solicitó cambios significativos y manifestó su satisfacción con los avances realizados. El feedback general fue muy positivo, lo que nos da confianza para continuar con las implementaciones planificadas.

Retrospectiva del sprint

¿Qué salió bien?

Se avanzó bastante en funciones importantes. Se logró implementar todo lo relacionado con la recuperación de contraseñas, el CRUD completo de usuarios tanto en el backend como en el frontend, y se incluyó un sistema de roles que ayuda a controlar el acceso a distintas rutas del sistema.

También hubo buena comunicación entre los miembros del equipo, lo que ayudó a resolver bloqueos rápidamente, sobre todo en la parte de integración entre frontend y backend. Además, se organizó bien el backlog y se priorizaron las tareas según su dependencia con otras.

¿Qué se puede mejorar?

Una de las cosas que se debe mejorar es cómo se distribuye el trabajo. Varias tareas complejas se dejaron para el final, justo cuando había menos disponibilidad por los días festivos. También sería bueno dividir las tareas grandes en partes más pequeñas, para que sea más fácil avanzar y dar seguimiento. Por último, sería útil empezar a hacer pruebas de integración desde el inicio, para evitar tener que corregir cosas al final.

¿Qué no salió tan bien?

La Semana Santa afectó el ritmo de trabajo y eso se notó en la acumulación de tareas al final. Algunas cosas importantes, como la asignación dinámica de roles, ni siquiera se empezaron. Además, algunas conexiones entre el frontend y el backend quedaron sin depurar, así que todavía habrá que trabajar en eso durante el siguiente sprint.

Excel con control de tiempos

→ [Horario software.xlsx](#)

Sistema de control:

→ [Link al jira](#)

Anexos

Figura 1: Login



The login form is centered on a light gray background. It features the GENSER logo at the top, which includes a sneaker and a high-heeled shoe. Below the logo is the text "COMERCIALIZADORA E IMPORTADORA". The form title is "Iniciar Sesión". It contains two input fields: "Usuario:" and "Contraseña:". A green "Ingresar" button is positioned below the password field. A link "Recuperar contraseña" is located at the bottom of the form.

GENSER
COMERCIALIZADORA E IMPORTADORA

Iniciar Sesión

Usuario:

Contraseña:

Ingresar

[Recuperar contraseña](#)

Figura 2: UI recuperación de contraseña



The password recovery form is centered on a light gray background. It features the GENSER logo at the top, which includes a sneaker and a high-heeled shoe. Below the logo is the text "COMERCIALIZADORA E IMPORTADORA". The form title is "Recuperar Contraseña". It includes the instruction "Ingrese el correo asociado al usuario". There is one input field labeled "Correo electrónico:". A green "Enviar" button is positioned below the input field. A link "Volver al login" is located at the bottom of the form.

GENSER
COMERCIALIZADORA E IMPORTADORA

Recuperar Contraseña

Ingrese el correo asociado al usuario

Correo electrónico:

Enviar

[Volver al login](#)

Figura 3: UI gestión de usuarios

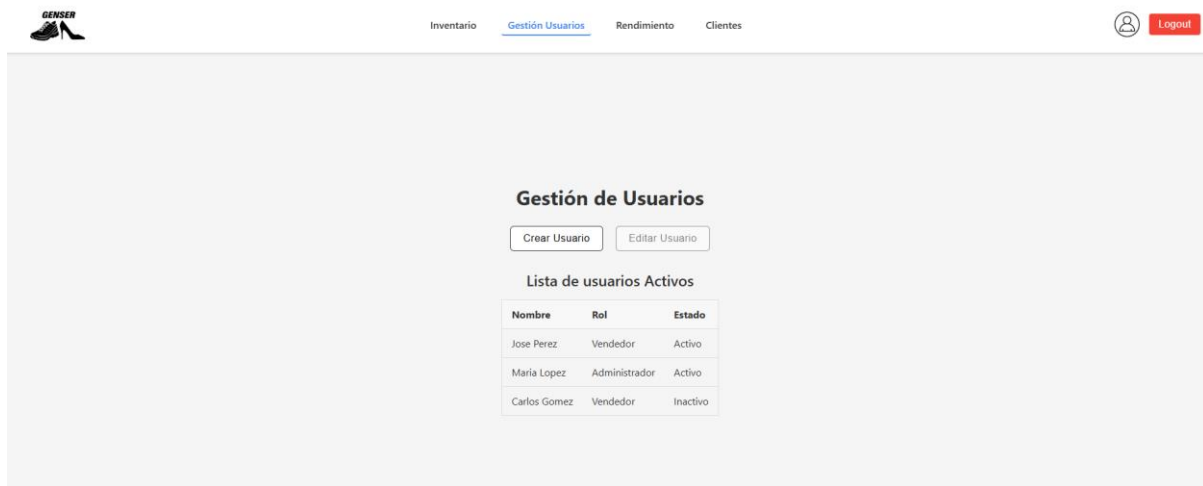


Figura 4: Formulario crear usuario

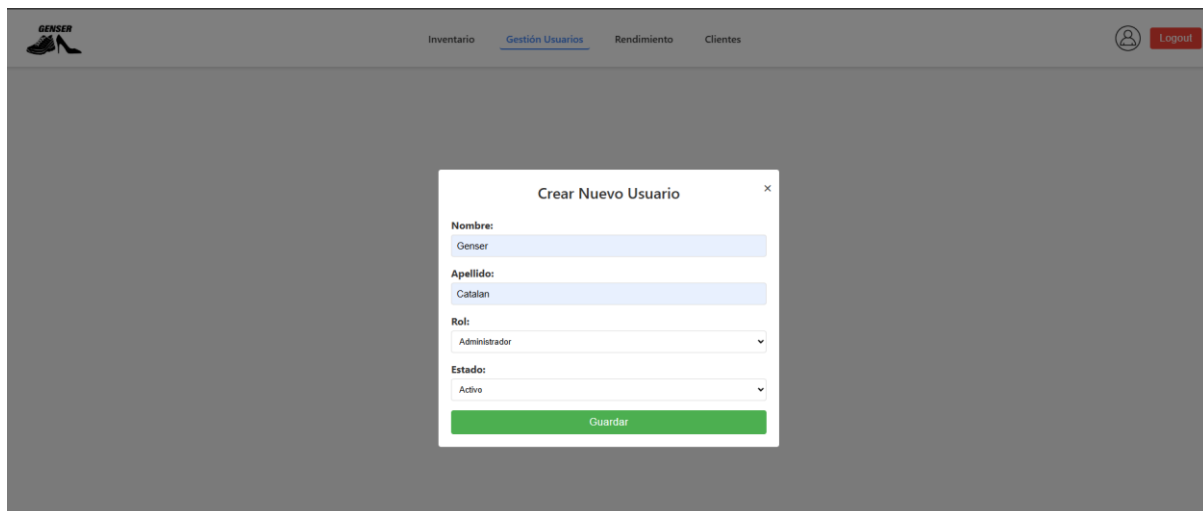


Figura 5: Formulario editar usuario

[Inventario](#)[Gestión Usuarios](#)[Rendimiento](#)[Clientes](#)[Logout](#)

Editar Usuario

Nombre:

Apellido:

Rol:

Estado:

Actualizar